
Snake project

Matteo Bino - 2158293

Abstract

The game of Snake provides a compelling environment for Reinforcement Learning due to its simple rules yet complex strategic requirements. As the snake grows, the risk of self-collision increases, requiring the agent to balance short-term gains with long-term survival. In this work, we investigate the transition from fully observable, grid-dependent states to partially observable, ego-centric representations. By utilizing a ray-casting visibility system, we decouple the input features from the absolute coordinates of the board. Additionally, we tackle the methodological hurdle of comparing agents trained under different reward functions. Since reward magnitudes are often arbitrary and scale-dependent, we propose an evaluation framework based on objective game-based metrics to provide a fair comparison between learning-based models and non-learning baselines.

1. Code

The local environment was run using python 3.12.12 with modules versions specified in the file `requirements.txt`. It is possible to reproduce the same environment by positioning in the main folder, then executing these commands (they require conda):

```
conda create -n snake_rl python=3.12.12 -y
conda activate snake_rl
python -m pip install --upgrade pip
pip install -r requirements.txt
```

It is possible to train all the models (see Section 5) by launching the script `train.py`. If not, only the 2 best models are kept, as they are needed in the evaluation script `evaluate.py`, that will evaluate baselines and models as specified in Section 6.2. These two scripts are also available in notebook format with markdown comments. Models and performances files are available in the folder `results`, divided between fully and partially observable environments.

2. Grid-Size-Invariance and Environment Visibility

Traditionally, Deep Reinforcement Learning models for grid-based games take the entire board as a multi-channel input. However, this couples the neural network architecture to a specific spatial resolution ($H \times W$), requiring retraining if the grid size changes. To overcome this limitation, we implemented an egocentric ray-casting system where the agent perceives the world through K rays pointing outwards from its head.

2.1. Ray-Casting Encoding

The encoder abstracts the grid state into a fixed-length feature vector of size $3K$. This is achieved by projecting K rays uniformly across 360° , centered at the snake's head coordinates. For every step, the encoder calculates the distance to three specific entity classes along each ray:

1. wall detection: by calculating the intersection of each directional ray vector with the four grid edges, the system identifies the minimum distance to a collision with a wall.
2. object mapping and binning: for discrete entities such as fruit and the snake's own body segments, the encoder converts their absolute grid coordinates into polar coordinates relative to the head. Each object is assigned to the "bin" of the ray whose angle most closely matches the object's relative angle.
3. proximity prioritization: if multiple objects of the same type fall within the angular scope of a single ray, the encoder retains only the distance to the closest one. This ensures the agent prioritizes immediate obstacles.

2.2. Feature Scaling and Representation

To ensure numerical stability and focus the agent's attention on immediate threats, distances are not fed into the model as raw integers. Instead, we utilize the *multiplicative inverse* of the distance ($1/d$). This transformation bounds all input features between 0 and 1. An immediate collision results in a value of 1.0, while objects very far away approach 0. The model becomes more sensitive to changes in distance when an object is close than when it is far away.

2.3. From Full to Partial Observability

A significant advantage of achieving scale invariance through ray-casting is that it provides an immediate bridge between fully observable and partially observable environments. In a traditional grid-based setup, switching from fully observable to partially observable would require a fundamental change in input shape, rendering fully observable-trained models useless in partially observable settings.

By contrast, the ray-casting encoder provides a consistent input dimensionality and feature representation across both observability regimes. In a fully observable environment, the rays extend to the boundaries of the board; in a partially observable environment, the rays are truncated by the limited visibility radius. Because the input features consist of normalized egocentric distances rather than global spatial coordinates, the semantics of the observation space remain compatible. Consequently, this approach facilitates robust knowledge transfer, allowing models optimized in a fully observable environment to be directly deployed in partially observable configurations and vice versa.

3. Reward Functions and Evaluation Strategy

3.1. Classic and Custom reward functions

Classic reward function: utilizes a static approach. It provides a positive reward for eating fruit and a fixed negative penalty for hitting a wall or colliding with its own body. Crucially, the step reward is zero. Under this regime, the agent is not penalized for taking long, circuitous paths to reach food, which can lead to inefficient movement patterns. Additionally, the penalty for self-collision is constant, regardless of whether the snake loses a single segment or a significant portion of its length.

Custom Reward Function: introduces a dense and dynamic structure designed to optimize the agent’s movement efficiency. It applies a dynamic step penalty where every move incurs a small negative reward that scales inversely with the snake’s relative length ($-0.05 \times \frac{1-\text{length}}{\text{board area}}$); this imposes strong pressure on smaller snakes to seek food while easing restrictions as spatial constraints naturally increase with body size. Furthermore, the self-collision penalty is made proportional to the number of body segments lost, providing a granular feedback signal against severe self-consumption ($-0.2 \times \frac{\text{body eaten}}{\text{previous length}}$). Finally, the penalty for hitting a wall is substantially increased (-0.4) and the fruit reward is lowered (0.4), explicitly prioritizing long-term survival as a prerequisite for maximizing the total score.

3.2. Standardized Evaluation Metrics

To provide an objective comparison between RL models—either using classic reward or custom reward—and non-

learning baselines, we established a suite of game metrics that are independent of the reward values.

- **Average apples eaten:** the primary performance metric. It measures the raw success of the agent in achieving the game’s objective.
- **Average body eaten:** a safety metric counting how many body segments were lost to self-collision.
- **Average death count:** tracks how many times the agent terminated by hitting a wall.
- **Efficiency ratios (apple, body, and death ratios):** by dividing the total counts of apples, body collisions, or deaths by the total number of steps taken, we normalize performance against survival time.

The distinction between absolute counts and ratios is vital, as an agent might achieve an high apple count simply by surviving for many steps through random movement, whereas the apple ratio can identify agents that are truly proficient at navigating toward the target.

4. Agents

4.1. Baselines

We implemented three distinct non-learning baseline agents. These baselines serve as performance benchmarks, representing different levels of strategic complexity. They do not use the ray-casted encoded grid, but directly the original grid of the environment.

- **Safe-Random Agent:** this agent serves as the simplest survival baseline. In each step, it identifies all immediate neighbors of the snake’s head and categorizes them into three priority tiers, here ordered by decreasing priority: *safe* (empty space or fruit), *body-collision* (hits its own body), and *terminal* (hits a wall). The agent picks a random action from the highest available tier. While this strategy avoids immediate death by wall collision, it lacks any objective-driven behavior, leading to low apple consumption and high survival variance.
- **Greedy-Objective Agent:** Building upon the safety logic of the Safe-Random agent, the Greedy agent introduces goal-oriented movement. It filters potential moves using the same safety tiers but, within the *safe* category, it selects the action that minimizes the Manhattan distance between the snake’s head and the fruit. This agent is highly efficient in the early game but frequently traps itself in later stages as it lacks the long-term foresight to leave room for its growing body.

- **Hamiltonian-Cycle Agent:** This agent utilizes a pre-computed deterministic path that visits every single tile on the grid exactly once before returning to the starting position. The implementation consists of two phases: a synchronization phase, where the agent uses a breadth-first Search to safely reach the cycle’s starting point, and a cycle-following phase. We implemented specific zig-zag patterns for both even and odd-sized grids to ensure the cycle is valid regardless of the grid dimensions: in case of odd-sized grids, one tile is left out in the classic Hamiltonian cycle. To avoid so, after the snake completes a cycle, during the next one it will visit the left-out cell. While this agent is guaranteed to win by filling the entire board, its path is highly inefficient, resulting in a very low apple-to-step ratio.

4.2. Deep Reinforcement Learning agents

The learning core of this project consists of a value-based and a policy-based architecture. Both agents process the flattened $3K$ feature vector provided by the ray-casting Encoder.

4.2.1. NEURAL NETWORK CONFIGURATION

Both agents utilize a Multi-Layer Perceptron as their function approximator. Specifically, it is composed of three hidden dense layers with 128, 64, and 32 neurons, respectively, activated by the swish activation function. The output layer is a linear dense layer tailored to the agent’s requirements (4 action Q -values/logits for the DQN or a single state value for the A2C).

4.2.2. DQN

The DQN agent was configured with a dual-network setup (live and target) to stabilize training. The exploration policy is the ϵ -greedy strategy initialized at 1.0, decaying by a factor of 0.995 to a minimum of 0.01. The target network weights are synchronized every 500 training steps.

4.2.3. ACTOR-CRITIC

The A2C agent also utilizes separate MLP heads for the Actor and the Critic, optimized simultaneously. We exploit entropy regularization (β) set to 0.01 to maintain policy diversity and prevent premature convergence to sub-optimal deterministic behaviors along normalized TD-error to stabilize the policy gradient updates.

5. Training Methodology

The training phase was designed to be highly efficient, leveraging massively parallel simulations to rapidly populate the experience replay buffer and stabilize the neural approximations.

5.1. Parallel Interaction and Experience Collection

To accelerate data collection, we utilized a training environment consisting of 1,000 parallel 10×10 boards. During each training iteration, the agent performs actions across all boards simultaneously. This parallelization ensures that every iteration provides a diverse set of transitions, covering a wide range of snake lengths and spatial configurations. The environment handles agent death by resetting only the specific boards where a collision occurred.

5.2. Replay Buffer

To break the temporal correlation of sequential data, transitions are stored in a pre-allocated experience replay buffer. The buffer configurations were specifically tuned for each agent to reflect their learning requirements:

- DQN buffer: optimized for off-policy learning with a capacity of 10,000 transitions. Training begins only after a minimum of 2,000 samples are collected. Each iteration performs 4 training epochs on a batch of 512 samples.
- A2C buffer: configured for a more on-policy style of updates with a smaller capacity of 1,000 transitions. It requires a full buffer before training and performs only 1 epoch per iteration on a batch of 512 samples, ensuring the policy is updated using fresh experiences as needed by the algorithm.

5.3. Observability Regimes

A primary goal of this study was to evaluate the robustness of the ray-casting system across different visibility conditions. Consequently, we trained four distinct models:

- DQN-FO & A2C-FO: Trained in a fully observable environment where the ray-caster has no range limit.
- DQN-PO & A2C-PO: Trained in a partially observable environment with a visibility radius of 2 units.

5.4. Optimization and Evaluation Loop

Training was conducted over 10,000 iterations using a discount factor of $\gamma = 0.99$. Optimization is done via Adam optimizer with learning rate 0.0001. Every 100 iterations, training statistics (loss and reward) were logged. Every 500 iterations, a formal evaluation was performed using a separate test environment. During evaluation, the agent’s performance was tracked using the metrics defined in Section 3.2. To ensure that we retained the most capable version of each agent, the system implemented a *best model* check: if the current evaluation’s average reward exceeded the historical maximum, the model was saved. This prevents the

final results from being affected by potential catastrophic forgetting or policy degradation during the later stages of training.

6. Results and Discussion

6.1. Training performances

In this section we analyze the training performances of the models with respect to the 0 flat line of the baselines. The main comparison is about average loss and reward evolution, but also an evolution of game statistics is discussed (see Figure 1).

6.1.1. FULLY OBSERVABLE ENVIRONMENT

The loss and reward evolution highlight the technical challenges of the Actor-Critic architecture in this environment: Both DQN variants (Classic and Custom) maintain a very low and stable loss, while the Actor-Critic models show significantly higher loss values and high volatility. This confirms that the policy-gradient updates were much noisier than the DQNs' value-based updates, leading to the observed instability in survival at the end of the training period. The average apples plot shows a dominant performance by the DQN-Classic model (red curve). It successfully reached a plateau of approximately 25 apples per episode, indicating a high level of proficiency in navigating the board. The Actor-Critic-Classic (blue curve) settled at a lower performance level of approximately 10 apples per episode. Crucially, both models utilizing the custom reward (green and orange curves) failed to eat any apples. This suggests that the dense negative step penalties and body-eaten penalties in the custom function were likely too high relative to the fruit reward, causing the agents to prioritize moving in safe loops to avoid penalties rather than exploring to find food. In conclusion, for the fully observable environment, the DQN-Classic configuration emerged as the only model to achieve high-level play.

6.1.2. PARTIALLY OBSERVABLE ENVIRONMENT

The learning dynamics in the partial observable setting are characterized by extreme instability, particularly for the value-based models: The DQN-Custom model (blue) exhibits massive spikes in loss, reaching values above 2.0. This indicates severe divergence in the Q -value estimations, as the agent receives conflicting feedback from an environment it cannot fully perceive. Both Actor-Critic models (orange and red) maintain a stable loss of approximately 0.5. While they are more stable than the DQN variants, this stability does not translate into high apple counts, confirming that the policy gradient approach is less effective than the classic value-based approach at handling the specific sparse-reward nature of this task. DQN-Classic (green

curve) achieved the highest average apples count of approximately 0.8 per episode. However, this performance is highly volatile, indicating that the value-based agent struggles to find a stable policy under high uncertainty. Similar to the fully observable environment results, both models utilizing custom rewards (blue and red curves) failed to eat any apples. In conclusion, the partially observable environment results highlight the limitations of local-only visibility. While the DQN-Classic remains the most capable of finding fruit, it is unable to replicate its fully observable environment performance.

6.2. Evaluation performances

Evaluation is done by simulating 1000 games in parallel on different grid sizes (10×10 , 20×20), keeping track of the per-episode averages of reward, number of apples eaten, number of body pieces eaten, deaths count. The same values are described also by calculating the ratio with the steps taken.

6.2.1. FULLY OBSERVABLE ENVIRONMENT

In the 10×10 fully observable environment, the greedy baseline achieves the highest efficiency (14.55% apple/step ratio), though it suffers from the highest self-collision rate (21.62 body pieces). The DQN model shows high competitiveness, reaching an apple/step ratio of 13.44%, which is nearly equivalent to the greedy agent but with much safer movement (only 0.99 body pieces eaten). This indicates that the DQN has learned to prioritize long-term survival over immediate path minimization. As the environment scales to 20×20 , the DQN exhibits superior scalability compared to the other learning model. While the A2C model effectively collapses, the DQN maintains a strong performance with 88.06 apples per episode, significantly outperforming the hamiltonian baseline in this specific scale. This suggests that the DQN's value-function approximation generalizes effectively to larger state spaces when global information is available. Tables 1, 2 show the complete performances of the agents.

6.2.2. PARTIALLY OBSERVABLE ENVIRONMENT

The transition to the partially observable environment represents a catastrophe for most agents. Random, greedy, and A2C agents suffer from extremely short lifespans (respectively, 17 and 29 and 109 steps in 10×10 grid), as they lack the global context to avoid immediate terminal collisions. The DQN model emerges as the only learning-based agent capable of sustained survival in partially observable environments, reaching the 2000-step limit in both 10×10 and 20×20 grids. While its absolute apple count is low (0.82 in 10×10), its death ratio per step is low (0.27%) compared to the Greedy agent (17.57%). This highlights the DQN's



Figure 1. Training performances across fully observable and partially observable environments. Plots from top to bottom: average losses, average rewards, average apples eaten, average body pieces eaten, average deaths. Averages are per episode.

Table 1. Average (per episode) agents performances on grid size 10 on fully observable environment.

Agent	Steps	Reward	Apples	Body Pieces	Deaths	Apples/steps %	Body Pieces/steps %	Deaths/steps %
Random	2000	11.03	24.21	5.37	0	1.21	0.27	0
Hamiltonian	2000	56.23	109.97	0	0	5.5	0	0
Greedy	2000	141.22	291.09	21.62	0	14.55	1.08	0
DQN	1514	17.3	35.2	0.99	1	13.44	0.38	0.38
A2C	2000	6.49	13.27	0.36	0.73	1.2	0.05	0.11

Table 2. Average (per episode) agents performances on grid size 20 on fully observable environment.

Agent	Steps	Reward	Apples	Body Pieces	Deaths	Apples/steps %	Body Pieces/steps %	Deaths/steps %
Random	2000	1.51	3.17	0.36	0	0.16	0.02	0
Hamiltonian	2000	6.16	12.31	0	0	0.62	0	0
Greedy	2000	68.73	139.42	4.89	0	6.97	0.24	0
DQN	2000	43.64	88.06	1.76	0.36	5.2	0.1	0.02
A2C	2000	1.17	2.34	0	0.1	0.12	0	~ 0

Table 3. Average (per episode) agents performances on grid size 10 on partially observable environment.

Agent	Steps	Reward	Apples	Body Pieces	Deaths	Apples/steps %	Body Pieces/steps %	Deaths/steps %
Random	109	-0.05	0.1	0	1	0.83	0	7.61
Hamiltonian	2000	56.23	109.97	0	0	5.5	0	0
Greedy	29	0.18	0.59	0.07	1	10.3	1.3	17.57
DQN	2000	0.33	0.82	0.001	0.85	0.26	~0	0.27
A2C	17	0.17	0.54	0	1	9.21	0	17.02

Table 4. Average (per episode) agents performances on grid size 20 on partially observable environment.

Agent	Steps	Reward	Apples	Body Pieces	Deaths	Apples/steps %	Body Pieces/steps %	Deaths/steps %
Random	538	-0.06	0.08	0	1	0.16	0	1.9
Hamiltonian	2000	6.16	12.31	0	0	0.62	0	0
Greedy	22	-0.004	0.2	0.01	1	2.05	0.1	10.39
DQN	2000	0.001	0.2	0	0.96	0.24	0	1.17
A2C	31	-0.01	0.18	0	1	1.75	0	9.82

ability to develop a conservative survival policy. A similar behaviour is shown for the grid size of 20. Tables 3, 4 show the complete performances of the agents.

6.3. Final Considerations

The experimental results demonstrate that value-based Reinforcement Learning provides a robust and scalable solution to the Snake game.

In fully observable environments, the DQN demonstrated an ability to match the efficiency of greedy search while significantly reducing the incidence of self-collision, effectively learning that long-term survival is a prerequisite for a high cumulative score. The scalability of the model was further validated in the 20×20 grid, where the DQN maintained a high level of proficiency while the A2C model failed to generalize, confirming the superior stability of value-function approximation for this specific task.

The results in partially observable environments further distinguish the DQN as a highly resilient architecture. While

the information bottleneck of a 2-unit visibility radius led to a catastrophe for the Greedy and A2C agents, the DQN developed a successful conservative policy. By prioritizing obstacle avoidance and localized survival over risky objective pursuit, the DQN remained the only agent capable of consistently reaching the episode step limit under severe uncertainty.

In conclusion, this work highlights that while reward shaping and environmental visibility significantly influence agent behavior, the underlying choice of RL architecture remains paramount. The DQN agent, supported by an egocentric feature representation, proves to be a versatile solution capable of maintaining high performance and survival across varying arena scales and observability regimes.